

DigiNiwas API v1

Authenticated, Versioned Backend Reference
Requests, Responses, Flows and Code-vs-PDF Audit

Canonical base URL

<http://localhost:5000/api/v1>

Generated from the supplied backend source and MongoDB sample exports. Existing /api routes remain temporary aliases and return deprecation headers.

1. Integration contract

Use `/api/v1` for every new frontend integration. Send protected requests with `Authorization: Bearer JWT_TOKEN`. JSON endpoints use `Content-Type: application/json`; property and CMS image uploads use `multipart/form-data`.

Access class	Mounted operations	Rule
Public	45	Login, registration, OTP and public content only
Bearer token	102	Any valid application JWT
Admin Bearer	80	JWT role must be admin

Standard authentication errors

```
{
  "success": false,
  "message": "Not authorized. Token missing."
}

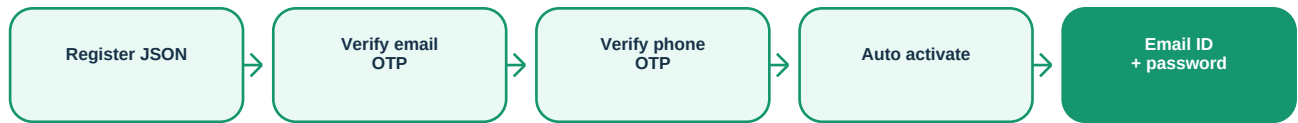
{
  "success": false,
  "message": "Invalid or expired token."
}

{
  "success": false,
  "message": "Admin access is required."
}
```

Version headers: canonical responses include `X-API-Version: 1`. Legacy responses include `Deprecation: true`, `Sunset`, and a `successor-version` Link header.

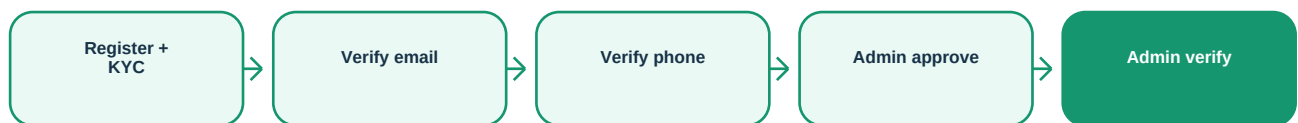
2. Backend business flows

Seller self-registration



No Aadhaar/PAN upload and no admin approval. Phone verification creates the temporary password, activates the seller, and sends credentials by email.

Partner onboarding



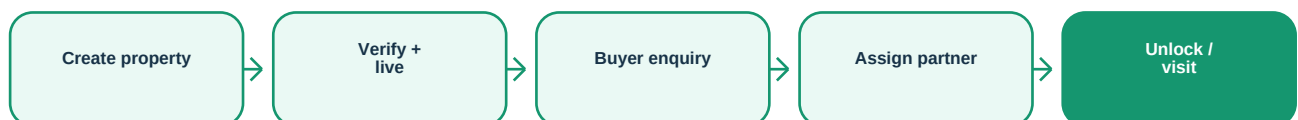
Partner registration retains identity documents and privacy consent. Approval issues temporary credentials; final verification enables permissions.

Buyer authentication



Buyer OTP login marks the phone verified and returns a JWT.

Property to lead to visit



Public discovery returns only Live + Verified properties. Lead generation follows property enquiry; partner unlock charges configured credits; visits move through team/admin review and status updates.

3. Existing PDF vs supplied code

The previous PDF lists 164 unique operations. The supplied/versioned code exposes 227 legacy-equivalent mounted operations. There are 8 PDF-only entries and 71 code-only entries.

In old PDF but not in current code

Method	Old documented path	Finding
DELETE	/api/properties/:id	Remove from frontend or restore intentionally
GET	/api/properties/dashboard/stats	Remove from frontend or restore intentionally
GET	/api/properties/search/list	Remove from frontend or restore intentionally
GET	/api/sellers/applications	Remove from frontend or restore intentionally
GET	/api/sellers/applications/:id	Remove from frontend or restore intentionally
PATCH	/api/sellers/:id/verify	Remove from frontend or restore intentionally
PATCH	/api/sellers/applications/:id/review	Remove from frontend or restore intentionally
PUT	/api/properties/:id	Remove from frontend or restore intentionally

In current code but missing from old PDF

Method	Current legacy alias
DELETE	/api/blog/admin/articles/:articleId
DELETE	/api/blog/admin/articles/:articleId/image
DELETE	/api/blog/admin/featured/image
DELETE	/api/blog/admin/hero/image
DELETE	/api/blog/admin/trending/:trendingId
DELETE	/api/cms/about-visionaries/member/:id
DELETE	/api/cms/properties/:id
DELETE	/api/properties/:id/delete
GET	/api/admin/dashboard/lead-analytics
GET	/api/admin/dashboard/summary
GET	/api/admin/notifications
GET	/api/admin/profile
GET	/api/agents/nearby
GET	/api/blog/admin
GET	/api/blog/admin/articles
GET	/api/blog/admin/articles/:articleId
GET	/api/blog/articles/slug/:slug
GET	/api/blog/public
GET	/api/cms/properties
GET	/api/cms/properties/:id
GET	/api/cms/properties/dashboard/stats
GET	/api/cms/properties/filter

Method	Current legacy alias
GET	/api/cms/properties/filter/list
GET	/api/cms/properties/search/list
GET	/api/credits/admin/adjustments
GET	/api/credits/partners/:partnerId
GET	/api/home/feed
GET	/api/leads/refund-requests
GET	/api/locations/popular
GET	/api/newproperties
GET	/api/newproperties/partner/:partnerId
GET	/api/promotions/my
GET	/api/properties/admin
GET	/api/properties/all
GET	/api/properties/boosted
GET	/api/properties/categories
GET	/api/properties/explore-nearby
GET	/api/properties/new-listings
GET	/api/properties/partner/:partnerId
GET	/api/user/dashboard-header
PATCH	/api/admin/notifications/:id/read
PATCH	/api/admin/notifications/read-all
PATCH	/api/admin/profile
PATCH	/api/admin/profile/change-password
PATCH	/api/blog/admin/articles/:articleId
PATCH	/api/blog/admin/categories
PATCH	/api/blog/admin/featured
PATCH	/api/blog/admin/hero
PATCH	/api/blog/admin/newsletter
PATCH	/api/blog/admin/niwas-ai
PATCH	/api/blog/admin/trending/:trendingId
PATCH	/api/cms/properties/:id/status
PATCH	/api/leads/:id/close
PATCH	/api/leads/:id/verify
PATCH	/api/leads/refund-requests/:requestId/review
PATCH	/api/partners/:id/block
PATCH	/api/properties/:id
PATCH	/api/properties/:id/boost
PATCH	/api/properties/:id/unboost
PATCH	/api/team-partners/:ownerId/members/:memberId/allocate-credits
POST	/api/blog/admin/articles

Method	Current legacy alias
POST	/api/blog/admin/trending
POST	/api/cms/properties
POST	/api/credits/admin/add
POST	/api/credits/admin/bulk-add
POST	/api/credits/admin/sync-wallets
POST	/api/credits/admin/withdraw
POST	/api/leads/:id/refund-request
POST	/api/promotions/partner/request
POST	/api/promotions/property/request
PUT	/api/cms/properties/:id

4. Endpoint index

Module	Operations	Public	Bearer	Admin
Admin	8	0	0	8
Agents	1	0	1	0
Auth	1	1	0	0
Auths	4	4	0	0
Blog	19	2	3	14
Boost Operations	2	0	0	2
Buyers	3	0	0	3
CMS - About Genesis	3	1	0	2
CMS - About Mission Vision	2	1	0	1
CMS - About Stats	2	1	0	1
CMS - About Visionaries	5	1	0	4
CMS - Agent Network	5	1	0	4
CMS - Agentcorner	2	1	0	1
CMS - Hero	3	1	0	2
CMS - Network Density	2	1	0	1
CMS - Plan Section	2	1	0	1
CMS - Platform Features	2	1	0	1
CMS - Properties	10	6	0	4
CMS - Round Section	3	1	0	2
CMS - Testimonials	5	2	0	3
Credit Settings	4	0	1	3
Credits	15	0	15	0
Home	1	0	1	0
Leads	21	0	21	0
Locations	1	0	1	0
Newproperties	12	3	9	0
Partner Applications	10	4	0	6
Partner Auth	2	1	1	0
Partners	12	0	0	12
Promotions	8	0	8	0
Properties	16	3	13	0
Property Publishing	5	0	0	5
Saved Properties	4	0	4	0
Sellers	14	8	6	0
Team Partners	8	0	8	0
User	1	0	1	0
Visits	9	0	9	0

5. Complete API catalog

All paths below are canonical and include /api/v1. Replace uppercase placeholders and :pathParams with real IDs. Success samples are representative controller envelopes; returned resource fields follow the corresponding schema/sample record.

Admin

GET /api/v1/admin/dashboard/lead-analytics

Admin Bearer

Purpose: Get lead analytics. (getLeadAnalytics)

Query params: period

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Admin/dashboardRoutes.js -> controllers/Admin/dashboardController.js

GET /api/v1/admin/dashboard/summary

Admin Bearer

Purpose: Get admin dashboard summary. (getAdminDashboardSummary)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Admin/dashboardRoutes.js -> controllers/Admin/dashboardController.js

GET /api/v1/admin/notifications

Admin Bearer

Purpose: Get admin notifications. (getAdminNotifications)

Query params: isRead, limit, page, type, unread

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Admin/notificationRoutes.js -> controllers/Admin/notificationController.js

PATCH /api/v1/admin/notifications/:id/read

Admin Bearer

Purpose: Mark admin notification read. (markAdminNotificationRead)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Mark admin notification read",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Admin/notificationRoutes.js -> controllers/Admin/notificationController.js

PATCH /api/v1/admin/notifications/read-all

Admin Bearer

Purpose: Mark all admin notifications read. (markAllAdminNotificationsRead)

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Mark all admin notifications read",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Admin/notificationRoutes.js -> controllers/Admin/notificationController.js

GET /api/v1/admin/profile

Admin Bearer

Purpose: Get admin profile. (getAdminProfile)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Admin/profileRoutes.js -> controllers/Admin/profileController.js

PATCH /api/v1/admin/profile

Admin Bearer

Purpose: Update admin profile. (updateAdminProfile)

Request body:

```
{
  "name": "Chhavi Chaturvedi",
  "phone": "9876543210",
  "profileImage": "https://example.com/profile.jpg"
}
```

Success response:

```
{
  "success": true,
  "message": "Update admin profile",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Admin/profileRoutes.js -> controllers/Admin/profileController.js

Purpose: Change admin password. (changeAdminPassword)

Request body:

```
{
  "currentPassword": "OldPassword@123",
  "newPassword": "NewPassword@123"
}
```

Success response:

```
{
  "success": true,
  "message": "Change admin password",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Admin/profileRoutes.js -> controllers/Admin/profileController.js

Agents

GET /api/v1/agents/nearby

Bearer token

Purpose: Get nearby agents. (getNearbyAgents)

Query params: limit

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/V1/publicAppRoutes.js -> controllers/V1/homeController.js

Auth

POST /api/v1/auth/login

Public

Purpose: Login. (login)

Request body:

```
{
  "email": "admin@diginiwas.com",
  "password": "Admin@123"
}
```

Success response:

```
{
  "success": true,
  "message": "Login successful",
  "token": "JWT_TOKEN",
  "data": {
    "id": "MONGODB_OBJECT_ID",
    "role": "buyer|seller|partner|admin"
  }
}
```

Source: routes/cms/authRoutes.js -> controllers/cms/authController.js

Auths

POST /api/v1/auths/login-otp

Public

Purpose: Login with otp. (loginWithOTP)

Request body:

```
{
  "phone": "9876543210",
  "otp": "123456",
  "role": "buyer"
}
```

Success response:

```
{
  "success": true,
  "message": "Login successful",
  "token": "JWT_TOKEN",
  "data": {
    "id": "MONGODB_OBJECT_ID",
    "role": "buyer|seller|partner|admin"
  }
}
```

Source: routes/authRoutes.js -> controllers/authController.js

POST /api/v1/auths/login-password

Public

Purpose: Login with password. (loginWithPassword)

Request body:

```
{
  "phone": "9876543210",
  "password": "Buyer@123",
  "role": "buyer"
}
```

Success response:

```
{
  "success": true,
  "message": "Login successful",
  "token": "JWT_TOKEN",
  "data": {
    "id": "MONGODB_OBJECT_ID",
    "role": "buyer|seller|partner|admin"
  }
}
```

Source: routes/authRoutes.js -> controllers/authController.js

POST /api/v1/auths/register

Public

Purpose: Register. (register)**Request body:**

```
{
  "name": "Rahul Sharma",
  "email": "rahul@example.com",
  "phone": "9876543210",
  "password": "Buyer@123",
  "role": "buyer",
  "location": {
    "city": "Mumbai",
    "state": "Maharashtra",
    "country": "India",
    "address": "Andheri West",
    "latitude": 19.1197,
    "longitude": 72.8464
  }
}
```

Success response:

```
{
  "success": true,
  "message": "Register",
  "data": "created or updated resource / operation result"
}
```

Source: routes/authRoutes.js -> controllers/authController.js**POST /api/v1/auths/send-otp**

Public

Purpose: Send otp. (sendOTP)**Request body:**

```
{
  "phone": "9876543210",
  "role": "buyer"
}
```

Success response:

```
{
  "success": true,
  "message": "OTP sent successfully",
  "data": {
    "phone": "9876543210",
    "otp": "123456"
  }
}
```

Source: routes/authRoutes.js -> controllers/authController.js

Blog

GET /api/v1/blog/admin

Bearer token

Purpose: Get blog page. (getBlogPage)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

GET /api/v1/blog/admin/articles

Bearer token

Purpose: Get all blog articles. (getAllBlogArticles)

Query params: category, published, search

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

POST /api/v1/blog/admin/articles

Admin Bearer

Purpose: Create blog article. (createBlogArticle)

Request body:

```
{
  "author": "<value>",
  "category": "Residential",
  "content": "<value>",
  "excerpt": "<value>",
  "featured": "<value>",
  "publishDate": "<value>",
  "published": "<value>",
  "readTime": "<value>",
  "sortOrder": "<value>",
  "title": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Create blog article",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

DELETE /api/v1/blog/admin/articles/:articleId

Admin Bearer

Purpose: Delete blog article. (deleteBlogArticle)

Path params: articleId

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

GET /api/v1/blog/admin/articles/:articleId

Bearer token

Purpose: Get blog article by id. (getBlogArticleById)

Path params: articleId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

PATCH /api/v1/blog/admin/articles/:articleId

Admin Bearer

Purpose: Update blog article. (updateBlogArticle)

Path params: articleId

Request body:

```
{
  "featured": "<value>",
  "publishDate": "<value>",
  "published": "<value>",
  "sortOrder": "<value>",
  "title": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update blog article",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

DELETE /api/v1/blog/admin/articles/:articleId/image

Admin Bearer

Purpose: Delete blog article image. (deleteBlogArticleImage)

Path params: articleId

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

PATCH /api/v1/blog/admin/categories

Admin Bearer

Purpose: Update blog categories. (updateBlogCategories)

Request body:

```
{
  "categories": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update blog categories",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

PATCH /api/v1/blog/admin/featured

Admin Bearer

Purpose: Update featured article. (updateFeaturedArticle)

Request body:

```
{
  "showAITake": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update featured article",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

DELETE /api/v1/blog/admin/featured/image

Admin Bearer

Purpose: Delete featured article image. (deleteFeaturedArticleImage)

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

PATCH /api/v1/blog/admin/hero

Admin Bearer

Purpose: Update blog hero. (updateBlogHero)**Request body:**

```
{
  "aiTag": "<value>",
  "buttonLink": "<value>",
  "buttonText": "<value>",
  "buyerTag": "<value>",
  "description": "<value>",
  "eyebrow": "<value>",
  "headingLine1": "<value>",
  "headingLine2": "<value>",
  "investmentTag": "<value>",
  "marketTag": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update blog hero",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js**DELETE** /api/v1/blog/admin/hero/image

Admin Bearer

Purpose: Delete blog hero image. (deleteBlogHeroImage)**Request body:** None.**Success response:**

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js**PATCH** /api/v1/blog/admin/newsletter

Admin Bearer

Purpose: Update blog newsletter. (updateBlogNewsletter)**Request body:**

```
{
  "enabled": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update blog newsletter",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

PATCH /api/v1/blog/admin/niwas-ai

Admin Bearer

Purpose: Update blog niwas ai. (updateBlogNiwasAi)

Request body:

```
{
  "buttonLink": "<value>",
  "buttonText": "<value>",
  "description": "<value>",
  "heading": "<value>",
  "questions": "<value>",
  "title": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update blog niwas ai",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

POST /api/v1/blog/admin/trending

Admin Bearer

Purpose: Create trending article. (createTrendingArticle)

Request body:

```
{
  "articleId": "ARTICLE_ID",
  "category": "Residential",
  "link": "<value>",
  "sortOrder": "<value>",
  "title": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Create trending article",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

DELETE /api/v1/blog/admin/trending/:trendingId

Admin Bearer

Purpose: Delete trending article. (deleteTrendingArticle)

Path params: trendingId

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

PATCH /api/v1/blog/admin/trending/:trendingId

Admin Bearer

Purpose: Update trending article. (updateTrendingArticle)

Path params: trendingId

Request body:

```
{
  "sortOrder": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update trending article",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

GET /api/v1/blog/articles/slug/:slug

Public

Purpose: Get article by slug. (getArticleBySlug)

Path params: slug

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

GET /api/v1/blog/public

Public

Purpose: Get public blog page. (getPublicBlogPage)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/blogRoutes.js -> controllers/cms/blogController.js

Boost Operations

GET /api/v1/boost-operations/:id

Admin Bearer

Purpose: Get boost request by id. (getBoostRequestById)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Promotion/boostDashboardRoutes.js -> controllers/Promotion/boostDashboardController.js

GET /api/v1/boost-operations/dashboard

Admin Bearer

Purpose: Get boost operations dashboard. (getBoostOperationsDashboard)

Query params: \$or, _id, limit, page, partnerId, partnerMongold, promotionType, propertyId, propertyMongold, requestId, search, status

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Promotion/boostDashboardRoutes.js -> controllers/Promotion/boostDashboardController.js

Buyers

GET /api/v1/buyers

Admin Bearer

Purpose: Get all buyers. (getAllBuyers)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/buyer/buyerRoutes.js -> controllers/Buyer/buyer.js

GET /api/v1/buyers/:id

Admin Bearer

Purpose: Get buyer by id. (getBuyerById)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/buyer/buyerRoutes.js -> controllers/Buyer/buyer.js

GET /api/v1/buyers/:id/dashboard

Admin Bearer

Purpose: Get buyer dashboard. (getBuyerDashboard)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/buyer/buyerRoutes.js -> controllers/Buyer/buyer.js

CMS - About Genesis

GET /api/v1/cms/about-genesis

Public

Purpose: Get about genesis. (getAboutGenesis)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/aboutGenesisRoutes.js -> controllers/cms/aboutGenesisController.js

PUT /api/v1/cms/about-genesis

Admin Bearer

Purpose: Update about genesis. (updateAboutGenesis)

Request body:

```
{
  "heading": "<value>",
  "paragraph1": "<value>",
  "paragraph2": "<value>",
  "quote": "<value>",
  "topTitle": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update about genesis",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/aboutGenesisRoutes.js -> controllers/cms/aboutGenesisController.js

DELETE /api/v1/cms/about-genesis/image

Admin Bearer

Purpose: Delete about genesis image. (deleteAboutGenesisImage)

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/aboutGenesisRoutes.js -> controllers/cms/aboutGenesisController.js

CMS - About Mission Vision

GET /api/v1/cms/about-mission-vision

Public

Purpose: Get about mission vision. (getAboutMissionVision)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/aboutMissionVisionRoutes.js -> controllers/cms/aboutMissionVisionController.js

PUT /api/v1/cms/about-mission-vision

Admin Bearer

Purpose: Update about mission vision. (updateAboutMissionVision)

Request body:

```
{
  "mission": "<value>",
  "vision": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update about mission vision",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/aboutMissionVisionRoutes.js -> controllers/cms/aboutMissionVisionController.js

CMS - About Stats

GET /api/v1/cms/about-stats

Public

Purpose: Get about stats. (getAboutStats)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/aboutStatsRoutes.js -> controllers/cms/aboutStatsController.js

PUT /api/v1/cms/about-stats

Admin Bearer

Purpose: Update about stats. (updateAboutStats)

Request body:

```
{
  "stats": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update about stats",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/aboutStatsRoutes.js -> controllers/cms/aboutStatsController.js

CMS - About Visionaries

GET /api/v1/cms/about-visionaries

Public

Purpose: Get visionaries. (getVisionaries)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/aboutVisionaryRoutes.js -> controllers/cms/aboutVisionaryController.js

PUT /api/v1/cms/about-visionaries/header

Admin Bearer

Purpose: Update visionary header. (updateVisionaryHeader)

Request body:

```
{
  "description": "<value>",
  "designation": "<value>",
  "name": "Demo User",
  "sectionSubtitle": "<value>",
  "sectionTitle": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update visionary header",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/aboutVisionaryRoutes.js -> controllers/cms/aboutVisionaryController.js

POST /api/v1/cms/about-visionaries/member

Admin Bearer

Purpose: Add visionary. (addVisionary)

Request body:

```
{
  "description": "<value>",
  "designation": "<value>",
  "name": "Demo User",
  "sectionSubtitle": "<value>",
  "sectionTitle": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Add visionary",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/aboutVisionaryRoutes.js -> controllers/cms/aboutVisionaryController.js

DELETE /api/v1/cms/about-visionaries/member/:id

Admin Bearer

Purpose: Delete visionary. (deleteVisionary)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/aboutVisionaryRoutes.js -> controllers/cms/aboutVisionaryController.js

PUT /api/v1/cms/about-visionaries/member/:id

Admin Bearer

Purpose: Update visionary. (updateVisionary)

Path params: id

Request body:

```
{
  "description": "<value>",
  "designation": "<value>",
  "name": "Demo User"
}
```

Success response:

```
{
  "success": true,
  "message": "Update visionary",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/aboutVisionaryRoutes.js -> controllers/cms/aboutVisionaryController.js

CMS - Agent Network

GET /api/v1/cms/agent-network

Public

Purpose: Get agent network. (getAgentNetwork)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/agentNetworkRoutes.js -> controllers/cms/agentNetworkController.js

POST /api/v1/cms/agent-network/card

Admin Bearer

Purpose: Add card. (addCard)

Request body:

```
{
  "designation": "<value>",
  "name": "Demo User",
  "review": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Add card",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/agentNetworkRoutes.js -> controllers/cms/agentNetworkController.js

DELETE /api/v1/cms/agent-network/card/:id

Admin Bearer

Purpose: Delete card. (deleteCard)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/agentNetworkRoutes.js -> controllers/cms/agentNetworkController.js

Purpose: Update card. (updateCard)

Path params: id

Request body:

```
{
  "designation": "<value>",
  "name": "Demo User",
  "review": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update card",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/agentNetworkRoutes.js -> controllers/cms/agentNetworkController.js

Purpose: Update heading. (updateHeading)

Request body:

```
{
  "designation": "<value>",
  "heading": "<value>",
  "name": "Demo User",
  "review": "<value>",
  "topTitle": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update heading",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/agentNetworkRoutes.js -> controllers/cms/agentNetworkController.js

CMS - Agentcorner

GET /api/v1/cms/agentcorner

Public

Purpose: Get agent corner. (getAgentCorner)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/agentCornerRoutes.js -> controllers/cms/agentCornerController.js

PUT /api/v1/cms/agentcorner

Admin Bearer

Purpose: Update agent corner. (updateAgentCorner)

Request body:

```
{
  "stats": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update agent corner",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/agentCornerRoutes.js -> controllers/cms/agentCornerController.js

CMS - Hero

GET /api/v1/cms/hero

Public

Purpose: Get hero section. (getHeroSection)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/heroSectionRoutes.js -> controllers/cms/heroSectionController.js

POST /api/v1/cms/hero

Admin Bearer

Purpose: Save hero section. (saveHeroSection)

Request body:

```
{
  "happyCustomers": "<value>",
  "headingLine1": "<value>",
  "headingLine2": "<value>",
  "localitiesCovered": "<value>",
  "propertiesListed": "<value>",
  "verifiedAgents": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Save hero section",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/heroSectionRoutes.js -> controllers/cms/heroSectionController.js

DELETE /api/v1/cms/hero/image

Admin Bearer

Purpose: Delete hero image. (deleteHeroImage)

Request body:

```
{
  "imageNo": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/heroSectionRoutes.js -> controllers/cms/heroSectionController.js

CMS - Network Density

GET /api/v1/cms/network-density

Public

Purpose: Get network density. (getNetworkDensity)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/networkDensityRoutes.js -> controllers/cms/networkDensityController.js

PUT /api/v1/cms/network-density

Admin Bearer

Purpose: Update network density. (updateNetworkDensity)

Request body:

```
{
  "cities": "<value>",
  "description": "<value>",
  "headingHighlight": "<value>",
  "headingLine1": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update network density",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/networkDensityRoutes.js -> controllers/cms/networkDensityController.js

CMS - Plan Section

GET /api/v1/cms/plan-section

Public

Purpose: Get plan section. (getPlanSection)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/planSectionRoutes.js -> controllers/cms/planSectionController.js

PUT /api/v1/cms/plan-section

Admin Bearer

Purpose: Update plan section. (updatePlanSection)

Request body:

```
{
  "heading": "<value>",
  "plans": "<value>",
  "topTitle": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update plan section",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/planSectionRoutes.js -> controllers/cms/planSectionController.js

CMS - Platform Features

GET /api/v1/cms/platform-features

Public

Purpose: Get platform features. (getPlatformFeatures)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/Platformroutes.js -> controllers/cms/platformFeatureController.js

POST /api/v1/cms/platform-features

Admin Bearer

Purpose: Save platform features. (savePlatformFeatures)

Request body:

```
{
  "features": "<value>",
  "topTitle": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Save platform features",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/Platformroutes.js -> controllers/cms/platformFeatureController.js

CMS - Properties

GET /api/v1/cms/properties

Public

Purpose: Get all properties. (getAllProperties)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/propertyRoutes.js -> controllers/cms/propertyController.js

POST /api/v1/cms/properties

Admin Bearer

Purpose: Add property. (addProperty)

Request body:

```
{
  "amenities": "<value>",
  "floorPlan": "<value>",
  "reraCertificate": "<value>",
  "tags": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Add property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/propertyRoutes.js -> controllers/cms/propertyController.js

DELETE /api/v1/cms/properties/:id

Admin Bearer

Purpose: Delete property. (deleteProperty)

Path params: id

Query params: category, city, keyword, maxPrice, minPrice, transactionType

Request body:

```
{
  "status": "In_Progress"
}
```

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/propertyRoutes.js -> controllers/cms/propertyController.js

GET /api/v1/cms/properties/:id

Public

Purpose: Get property by id. (getPropertyById)

Path params: id

Query params: category, city, keyword, maxPrice, minPrice, transactionType

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/propertyRoutes.js -> controllers/cms/propertyController.js

PUT /api/v1/cms/properties/:id

Admin Bearer

Purpose: Update property. (updateProperty)

Path params: id

Query params: category, city, keyword, maxPrice, minPrice, transactionType

Request body:

```
{
  "status": "In_Progress"
}
```

Success response:

```
{
  "success": true,
  "message": "Update property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/propertyRoutes.js -> controllers/cms/propertyController.js

PATCH /api/v1/cms/properties/:id/status

Admin Bearer

Purpose: Change property status. (changePropertyStatus)

Path params: id

Query params: category, city, keyword, maxPrice, minPrice, transactionType

Request body:

```
{
  "status": "In_Progress"
}
```

Success response:

```
{
  "success": true,
  "message": "Change property status",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/propertyRoutes.js -> controllers/cms/propertyController.js

GET /api/v1/cms/properties/dashboard/stats

Public

Purpose: Property dashboard. (propertyDashboard)

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Property dashboard",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/propertyRoutes.js -> controllers/cms/propertyController.js

GET /api/v1/cms/properties/filter

Public

Purpose: Get all propertiesbyfilter. (getAllPropertiesbyfilter)

Query params: category, city, keyword, maxPrice, minPrice, propertyType, transactionType

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/propertyRoutes.js -> controllers/cms/propertyController.js

GET /api/v1/cms/properties/filter/list

Public

Purpose: Filter properties. (filterProperties)

Query params: category, city, maxPrice, minPrice, transactionType

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/propertyRoutes.js -> controllers/cms/propertyController.js

GET /api/v1/cms/properties/search/list

Public

Purpose: Search properties. (searchProperties)

Query params: category, city, keyword, maxPrice, minPrice, transactionType

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/propertyRoutes.js -> controllers/cms/propertyController.js

CMS - Round Section

GET /api/v1/cms/round-section

Public

Purpose: Get round section. (getRoundSection)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/roundSectionRoutes.js -> controllers/cms/roundSectionController.js

POST /api/v1/cms/round-section

Admin Bearer

Purpose: Save round section. (saveRoundSection)

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Save round section",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/roundSectionRoutes.js -> controllers/cms/roundSectionController.js

DELETE /api/v1/cms/round-section/image

Admin Bearer

Purpose: Delete round image. (deleteRoundImage)

Request body:

```
{
  "imageNo": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/roundSectionRoutes.js -> controllers/cms/roundSectionController.js

CMS - Testimonials

GET /api/v1/cms/testimonials

Public

Purpose: Get testimonials. (getTestimonials)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/testimonialRoutes.js -> controllers/cms/testimonialController.js

POST /api/v1/cms/testimonials

Admin Bearer

Purpose: Create testimonial. (createTestimonial)

Request body:

```
{
  "designation": "<value>",
  "name": "Demo User",
  "review": "<value>",
  "videoLink": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Create testimonial",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/testimonialRoutes.js -> controllers/cms/testimonialController.js

DELETE /api/v1/cms/testimonials/:id

Admin Bearer

Purpose: Delete testimonial. (deleteTestimonial)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/cms/testimonialRoutes.js -> controllers/cms/testimonialController.js

GET /api/v1/cms/testimonials/:id

Public

Purpose: Get single testimonial. (getSingleTestimonial)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/cms/testimonialRoutes.js -> controllers/cms/testimonialController.js

Purpose: Update testimonial. (updateTestimonial)

Path params: id

Request body:

```
{
  "designation": "<value>",
  "name": "Demo User",
  "review": "<value>",
  "videoLink": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update testimonial",
  "data": "created or updated resource / operation result"
}
```

Source: routes/cms/testimonialRoutes.js -> controllers/cms/testimonialController.js

Credit Settings

GET /api/v1/credit-settings

Bearer token

Purpose: Get credit settings. (getCreditSettings)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Credit/creditSettingRoutes.js -> controllers/Credit/creditSettingController.js

PATCH /api/v1/credit-settings

Admin Bearer

Purpose: Update credit settings. (updateCreditSettings)

Request body:

```
{
  "creditsPerRupee": "<value>",
  "products": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update credit settings",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Credit/creditSettingRoutes.js -> controllers/Credit/creditSettingController.js

PATCH /api/v1/credit-settings/product/:code

Admin Bearer

Purpose: Update credit product. (updateCreditProduct)

Path params: code

Request body:

```
{
  "credits": 100,
  "durationDays": "<value>",
  "isActive": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update credit product",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Credit/creditSettingRoutes.js -> controllers/Credit/creditSettingController.js

POST /api/v1/credit-settings/reset

Admin Bearer

Purpose: Reset credit settings. (resetCreditSettings)

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Reset credit settings",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Credit/creditSettingRoutes.js -> controllers/Credit/creditSettingController.js

Credits

PATCH /api/v1/credits/adjust

Bearer token

Purpose: Adjust partner credits. (adjustPartnerCredits)

Request body:

```
{
  "partnerId": "6a8d62970d52efcce682f351",
  "credits": 25,
  "direction": "CREDIT",
  "reason": "Balance correction"
}
```

Success response:

```
{
  "success": true,
  "message": "Adjust partner credits",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/creditController.js

POST /api/v1/credits/admin/add

Bearer token

Purpose: Add credits by admin. (addCreditsByAdmin)

Request body:

```
{
  "partnerId": "6a8d62970d52efcce682f351",
  "credits": 100,
  "creditBucket": "PROMOTIONAL",
  "reason": "Promotional bonus"
}
```

Success response:

```
{
  "success": true,
  "message": "Add credits by admin",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/adminCreditController.js

GET /api/v1/credits/admin/adjustments

Bearer token

Purpose: Get admin adjustments. (getAdminAdjustments)

Query params: \$or, limit, page, partnerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/adminCreditController.js

POST /api/v1/credits/admin/bulk-add

Bearer token

Purpose: Bulk add credits by admin. (bulkAddCreditsByAdmin)

Request body:

```
{
  "allPartners": "<value>",
  "amount": "<value>",
  "credits": 100,
  "partnerIds": "<value>",
  "reason": "Administrative adjustment",
  "reference": "<value>",
  "source": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Bulk add credits by admin",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/adminCreditController.js

POST /api/v1/credits/admin/sync-wallets

Bearer token

Purpose: Sync team wallets by admin. (syncTeamWalletsByAdmin)

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Sync team wallets by admin",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/adminCreditController.js

POST /api/v1/credits/admin/withdraw

Bearer token

Purpose: Withdraw credits by admin. (withdrawCreditsByAdmin)

Request body:

```
{
  "partnerId": "6a8d62970d52efcce682f351",
  "credits": 25,
  "reason": "Manual correction"
}
```

Success response:

```
{
  "success": true,
  "message": "Withdraw credits by admin",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/adminCreditController.js

GET /api/v1/credits/dashboard

Bearer token

Purpose: Get credit dashboard. (getCreditDashboard)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/creditController.js

GET /api/v1/credits/history

Bearer token

Purpose: Get credit history. (getCreditHistory)

Query params: direction, limit, page, partnerId, partnerMongoid, productCode, status, type

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/creditController.js

GET /api/v1/credits/partner/:partnerId

Bearer token

Purpose: Get partner wallet. (getPartnerWallet)

Path params: partnerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/creditController.js

GET /api/v1/credits/partners

Bearer token

Purpose: Get partner credit overview. (getPartnerCreditOverview)

Query params: limit, page, search

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/creditController.js

GET /api/v1/credits/partners/:partnerId

Bearer token

Purpose: Get partner credit details. (getPartnerCreditDetails)

Path params: partnerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/creditController.js

GET /api/v1/credits/properties

Bearer token

Purpose: Get property credit overview. (getPropertyCreditOverview)

Query params: limit, page, search

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/creditController.js

GET /api/v1/credits/properties/:propertyId

Bearer token

Purpose: Get property credit details. (getPropertyCreditDetails)

Path params: propertyId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/creditController.js

POST /api/v1/credits/purchase/complete

Bearer token

Purpose: Complete credit purchase. (completeCreditPurchase)

Request body:

```
{
  "partnerId": "6a8d62970d52efcce682f351",
  "amountInRupees": 500,
  "paymentId": "pay_demo_001",
  "orderId": "order_demo_001",
  "idempotencyKey": "purchase-rahul-001"
}
```

Success response:

```
{
  "success": true,
  "message": "Complete credit purchase",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/creditController.js

Purpose: Refund credits. (refundCredits)

Request body:

```
{
  "partnerId": "6a8d62970d52efcce682f351",
  "credits": 25,
  "relatedTransactionId": "TRANSACTION_ID",
  "reason": "Approved refund"
}
```

Success response:

```
{
  "success": true,
  "message": "Refund credits",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Credit/creditRoutes.js -> controllers/Credit/creditController.js

Home

GET /api/v1/home/feed

Bearer token

Purpose: Get home feed. (getHomeFeed)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/V1/publicAppRoutes.js -> controllers/V1/homeController.js

Leads

GET /api/v1/leads

Bearer token

Purpose: Get leads. (getLeads)

Query params: \$or, limit, page, partnerId, priority, search, source, status

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

POST /api/v1/leads

Bearer token

Purpose: Create lead. (createLead)

Request body:

```
{
  "createdBy": "<value>",
  "estimatedValue": "<value>",
  "message": "<value>",
  "priority": "<value>",
  "propertyId": "6a8e87c256ffd64b71414aa6",
  "source": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Create lead",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

GET /api/v1/leads/:id

Bearer token

Purpose: Get lead by id. (getLeadById)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

PATCH /api/v1/leads/:id/allocate-subpartner

Bearer token

Purpose: Allocate lead to sub partner. (allocateLeadToSubPartner)

Path params: id

Request body:

```
{
  "subPartnerId": "SUB_PARTNER_ID",
  "remarks": "Allocated by team owner"
}
```

Success response:

```
{
  "success": true,
  "message": "Allocate lead to sub partner",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

PATCH /api/v1/leads/:id/assign-partner

Bearer token

Purpose: Assign lead partner. (assignLeadPartner)

Path params: id

Request body:

```
{
  "actor": "<value>",
  "partnerId": "6a8d62970d52efcce682f351",
  "remarks": "Reviewed and approved"
}
```

Success response:

```
{
  "success": true,
  "message": "Assign lead partner",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

PATCH /api/v1/leads/:id/close

Bearer token

Purpose: Close lead. (closeLead)

Path params: id

Request body:

```
{
  "actor": "<value>",
  "reason": "Administrative adjustment"
}
```

Success response:

```
{
  "success": true,
  "message": "Close lead",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

POST /api/v1/leads/:id/contact-history

Bearer token

Purpose: Add lead contact history. (addLeadContactHistory)

Path params: id

Request body:

```
{
  "contactedAt": "<value>",
  "doneBy": "<value>",
  "notes": "<value>",
  "status": "In_Progress",
  "type": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Add lead contact history",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

PATCH /api/v1/leads/:id/convert

Bearer token

Purpose: Convert lead. (convertLead)

Path params: id

Request body:

```
{
  "actor": "<value>",
  "amount": "<value>",
  "notes": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Convert lead",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

POST /api/v1/leads/:id/refund-request

Bearer token

Purpose: Create lead refund request. (createLeadRefundRequest)

Path params: id

Request body:

```
{
  "details": "<value>",
  "reason": "Administrative adjustment"
}
```

Success response:

```
{
  "success": true,
  "message": "Create lead refund request",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadRefundRequestController.js

PATCH /api/v1/leads/:id/reject

Bearer token

Purpose: Reject lead. (rejectLead)**Path params:** id**Request body:**

```
{
  "actor": "<value>",
  "reason": "Administrative adjustment"
}
```

Success response:

```
{
  "success": true,
  "message": "Reject lead",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js**PATCH** /api/v1/leads/:id/remove-subpartner-allocation

Bearer token

Purpose: Remove lead sub partner allocation. (removeLeadSubPartnerAllocation)**Path params:** id**Request body:**

```
{
  "remarks": "Reviewed and approved",
  "teamPartnerId": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Remove lead sub partner allocation",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js**PATCH** /api/v1/leads/:id/review

Bearer token

Purpose: Review lead. (reviewLead)**Path params:** id**Request body:**

```
{
  "actor": "<value>",
  "notes": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Review lead",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

PATCH /api/v1/leads/:id/status

Bearer token

Purpose: Update lead status. (updateLeadStatus)

Path params: id

Request body:

```
{
  "actor": "<value>",
  "remarks": "Reviewed and approved",
  "status": "In_Progress"
}
```

Success response:

```
{
  "success": true,
  "message": "Update lead status",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

PATCH /api/v1/leads/:id/unlock

Bearer token

Purpose: Unlock lead by partner. (unlockLeadByPartner)

Path params: id

Request body:

```
{
  "partnerId": "6a8d62970d52efcce682f351",
  "remarks": "Reviewed and approved"
}
```

Success response:

```
{
  "success": true,
  "message": "Unlock lead by partner",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

PATCH /api/v1/leads/:id/verify

Bearer token

Purpose: Verify lead by admin. (verifyLeadByAdmin)

Path params: id

Request body:

```
{
  "actor": "<value>",
  "notes": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Verify lead by admin",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

GET /api/v1/leads/dashboard

Bearer token

Purpose: Get lead dashboard. (getLeadDashboard)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

POST /api/v1/leads/from-property

Bearer token

Purpose: Create lead from property. (createLeadFromProperty)

Request body:

```
{
  "buyerId": "6a86b08e6007083b597138c1",
  "propertyId": "6a8ead9b58a6bb8fab38ebc1",
  "enquiryMessage": "I am interested in this property.",
  "consentVerified": true
}
```

Success response:

```
{
  "success": true,
  "message": "Create lead from property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

GET /api/v1/leads/partner/:partnerId

Bearer token

Purpose: Get partner visible leads. (getPartnerVisibleLeads)

Path params: partnerId

Query params: \$or, limit, page, search, status

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

GET /api/v1/leads/partner/:partnerId:id

Bearer token

Purpose: Get partner visible lead by id. (getPartnerVisibleLeadById)

Path params: id, partnerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadcontroller.js

Purpose: Get lead refund requests. (getLeadRefundRequests)

Query params: status

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadRefundRequestController.js

Purpose: Review lead refund request. (reviewLeadRefundRequest)

Path params: requestId

Request body:

```
{
  "remarks": "Reviewed and approved",
  "status": "In_Progress"
}
```

Success response:

```
{
  "success": true,
  "message": "Review lead refund request",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Lead/leadroutes.js -> controllers/Lead/leadRefundRequestController.js

Locations

GET /api/v1/locations/popular

Bearer token

Purpose: Get popular locations. (getPopularLocations)

Query params: limit

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/V1/publicAppRoutes.js -> controllers/V1/homeController.js

Newproperties

GET /api/v1/newproperties

Public

Purpose: Get filtered live properties. (getFilteredLiveProperties)

Query params: \$or, bathrooms, bedrooms, category, city, furnishing, limit, maxPrice, minPrice, negotiable, page, price, propertyType, search, transactionType

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

POST /api/v1/newproperties

Bearer token

Purpose: Create property. (createProperty)

Request body:

```
{
  "amenities": "<value>",
  "category": "Residential",
  "city": "Indore",
  "creatorId": "<value>",
  "creatorRole": "<value>",
  "negotiable": "<value>",
  "price": "<value>",
  "status": "In_Progress",
  "tags": "<value>",
  "title": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Create property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

GET /api/v1/newproperties/:id

Public

Purpose: Get admin property by id. (getAdminPropertyById)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

PATCH /api/v1/newproperties/:id

Bearer token

Purpose: Update property. (updateProperty)**Path params:** id**Request body:**

```
{
  "existingImages": "<value>",
  "status": "In_Progress",
  "statusRemarks": "<value>",
  "updatedByName": "<value>",
  "updatedByRole": "<value>",
  "updatedByUserId": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js**PATCH** /api/v1/newproperties/:id/boost

Bearer token

Purpose: Boost property. (boostProperty)**Path params:** id**Request body:**

```
{
  "boostType": "<value>",
  "days": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Boost property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js**DELETE** /api/v1/newproperties/:id/delete

Bearer token

Purpose: Delete property. (deleteProperty)**Path params:** id**Request body:** None.**Success response:**

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

PATCH /api/v1/newproperties/:id/status

Bearer token

Purpose: Update property status. (updatePropertyStatus)

Path params: id

Request body:

```
{
  "notes": "<value>",
  "rejectionReason": "<value>",
  "status": "In_Progress"
}
```

Success response:

```
{
  "success": true,
  "message": "Update property status",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

PATCH /api/v1/newproperties/:id/unboost

Bearer token

Purpose: Remove property boost. (removePropertyBoost)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Remove property boost",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

GET /api/v1/newproperties/admin

Bearer token

Purpose: Get admin properties. (getAdminProperties)

Query params: \$or, boosted, partnerId, search, status

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

GET /api/v1/newproperties/all

Bearer token

Purpose: Get all properties. (getAllProperties)

Query params: \$or, city, role, search, status

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

GET /api/v1/newproperties/filter

Public

Purpose: Get filtered live properties. (getFilteredLiveProperties)

Query params: \$or, bathrooms, bedrooms, category, city, furnishing, limit, maxPrice, minPrice, negotiable, page, price, propertyType, search, transactionType

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

GET /api/v1/newproperties/partner/:partnerId

Bearer token

Purpose: Get properties by partner. (getPropertiesByPartner)

Path params: partnerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

Partner Applications

GET /api/v1/partner-applications

Admin Bearer

Purpose: Get partner applications. (getPartnerApplications)

Query params: \$or, accountType, agencyOwnerId, applicationStatus, parentPartnerId, search, status

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/partnerApplicationRoutes.js -> controllers/Agent/partnerApplicationController.js

GET /api/v1/partner-applications/:id

Admin Bearer

Purpose: Get partner application by id. (getPartnerApplicationById)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/partnerApplicationRoutes.js -> controllers/Agent/partnerApplicationController.js

PATCH /api/v1/partner-applications/:id/action-required

Admin Bearer

Purpose: Request partner action. (requestPartnerAction)

Path params: id

Request body:

```
{
  "remarks": "Please provide a clearer identity document"
}
```

Success response:

```
{
  "success": true,
  "message": "Request partner action",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/partnerApplicationRoutes.js -> controllers/Agent/partnerApplicationController.js

PATCH /api/v1/partner-applications/:id/approve

Admin Bearer

Purpose: Approve partner application. (approvePartnerApplication)

Path params: id

Request body:

```
{
  "remarks": "Documents checked; application approved"
}
```

Success response:

```
{
  "success": true,
  "message": "Approve partner application",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/partnerApplicationRoutes.js -> controllers/Agent/partnerApplicationController.js

PATCH /api/v1/partner-applications/:id/reject

Admin Bearer

Purpose: Reject partner application. (rejectPartnerApplication)

Path params: id

Request body:

```
{
  "remarks": "Application details could not be verified"
}
```

Success response:

```
{
  "success": true,
  "message": "Reject partner application",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/partnerApplicationRoutes.js -> controllers/Agent/partnerApplicationController.js

POST /api/v1/partner-applications/:id/resend-otp

Public

Purpose: Resend partner otp. (resendPartnerOtp)

Path params: id

Request body:

```
{
  "channel": "phone"
}
```

Success response:

```
{
  "success": true,
  "message": "OTP resent successfully",
  "data": {
    "applicationId": "PARTNER_MONGODB_ID",
    "channel": "phone",
    "otp": "123456",
    "expiresIn": "10 minutes"
  }
}
```

Source: routes/Agent/partnerApplicationRoutes.js -> controllers/Agent/partnerApplicationController.js

PATCH /api/v1/partner-applications/:id/verify

Admin Bearer

Purpose: Mark partner verified. (markPartnerVerified)**Path params:** id**Request body:**

```
{
  "remarks": "Partner identity and business verified"
}
```

Success response:

```
{
  "success": true,
  "message": "Mark partner verified",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/partnerApplicationRoutes.js -> controllers/Agent/partnerApplicationController.js**PATCH** /api/v1/partner-applications/:id/verify-email

Public

Purpose: Verify partner email otp. (verifyPartnerEmailOtp)**Path params:** id**Request body:**

```
{
  "otp": "123456"
}
```

Success response:

```
{
  "success": true,
  "message": "Verify partner email otp",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/partnerApplicationRoutes.js -> controllers/Agent/partnerApplicationController.js**PATCH** /api/v1/partner-applications/:id/verify-phone

Public

Purpose: Verify partner phone otp. (verifyPartnerPhoneOtp)**Path params:** id**Request body:**

```
{
  "otp": "123456"
}
```

Success response:

```
{
  "success": true,
  "message": "Verify partner phone otp",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/partnerApplicationRoutes.js -> controllers/Agent/partnerApplicationController.js

Purpose: Register partner application. (registerPartnerApplication)

Request body:

```
{
  "name": "Rahul Partner",
  "email": "partner@example.com",
  "phone": "9876543211",
  "accountType": "single",
  "business": {
    "businessName": "Rahul Properties",
    "businessType": "Individual Broker",
    "experienceYears": 3
  },
  "location": {
    "city": "Indore",
    "state": "Madhya Pradesh",
    "country": "India",
    "address": "Vijay Nagar",
    "pinCode": "452010",
    "latitude": 22.7533,
    "longitude": 75.8937
  },
  "rera": {
    "applicable": true,
    "registrationNumber": "RERA-MP-123456"
  },
  "identityDocuments": [
    {
      "documentType": "AADHAAR",
      "frontUrl": "https://example.com/front.jpg",
      "backUrl": "https://example.com/back.jpg",
      "numberMasked": "XXXX-XXXX-1234"
    }
  ],
  "privacyConsent": {
    "accepted": true,
    "privacyNoticeVersion": "v1"
  }
}
```

Success response:

```
{
  "success": true,
  "message": "Register partner application",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/partnerApplicationRoutes.js -> controllers/Agent/partnerApplicationController.js

Partner Auth

PATCH /api/v1/partner-auth/change-password

Bearer token

Purpose: Change partner password. (changePartnerPassword)

Request body:

```
{
  "currentPassword": "<value>",
  "newPassword": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Change partner password",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Auth/partnerAuthRoutes.js -> controllers/Auth/partnerAuthController.js

POST /api/v1/partner-auth/login

Public

Purpose: Partner login. (partnerLogin)

Request body:

```
{
  "email": "rahulsharma@gmail.com",
  "password": "TEMP_PASSWORD_FROM_EMAIL "
}
```

Success response:

```
{
  "success": true,
  "message": "Login successful",
  "token": "JWT_TOKEN",
  "data": {
    "id": "MONGODB_OBJECT_ID",
    "role": "buyer|seller|partner|admin"
  }
}
```

Source: routes/Auth/partnerAuthRoutes.js -> controllers/Auth/partnerAuthController.js

Partners

GET /api/v1/partners

Admin Bearer

Purpose: Get all partners. (getAllPartners)

Query params: \$or, accountType, search

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

GET /api/v1/partners/:id

Admin Bearer

Purpose: Get partner by id. (getPartnerById)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

PATCH /api/v1/partners/:id/block

Admin Bearer

Purpose: Block partner. (blockPartner)

Path params: id

Request body:

```
{
  "isBlocked": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Block partner",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

GET /api/v1/partners/approved-not-verified

Admin Bearer

Purpose: Get approved not verified partners. (getApprovedNotVerifiedPartners)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

DELETE /api/v1/partners/delete/:id

Admin Bearer

Purpose: Delete partner. (deletePartner)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

GET /api/v1/partners/partners/available

Admin Bearer

Purpose: Get available partners. (getAvailablePartners)

Query params: accountType, city

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

GET /api/v1/partners/properties

Admin Bearer

Purpose: Get assignment properties. (getAssignmentProperties)

Query params: \$or, assignment

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

PATCH /api/v1/partners/properties/:propertyId/assign

Admin Bearer

Purpose: Assign partner to property. (assignPartnerToProperty)

Path params: propertyId

Request body:

```
{
  "partnerId": "6a8d62970d52efcce682f351"
}
```

Success response:

```
{
  "success": true,
  "message": "Assign partner to property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

PATCH /api/v1/partners/properties/:propertyId/assign-partner

Admin Bearer

Purpose: Assign partner to property. (assignPartnerToProperty)

Path params: propertyId

Request body:

```
{
  "partnerId": "6a8d62970d52efcce682f351"
}
```

Success response:

```
{
  "success": true,
  "message": "Assign partner to property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

PATCH /api/v1/partners/properties/:propertyId/unassign

Admin Bearer

Purpose: Unassign partner from property. (unassignPartnerFromProperty)

Path params: propertyId

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Unassign partner from property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

GET /api/v1/partners/properties/unassigned

Admin Bearer

Purpose: Get unassigned properties. (getUnassignedProperties)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

GET /api/v1/partners/summary

Admin Bearer

Purpose: Get assignment summary. (getAssignmentSummary)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/partnerRoutes.js -> controllers/Agent/partnerController.js

Promotions

GET /api/v1/promotions

Bearer token

Purpose: Get promotion requests. (getPromotionRequests)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Promotion/promotionRoutes.js -> controllers/Promotion/promotionV2Controller.js

POST /api/v1/promotions

Bearer token

Purpose: Create promotion request. (createPromotionRequest)

Query params: \$or, limit, page, partnerId, partnerMongoid, promotionType, propertyId, propertyMongoid, search, status, targetType

Request body:

```
{
  "idempotencyKey": "<value>",
  "locality": "<value>",
  "partnerId": "6a8d62970d52efcce682f351",
  "promotionType": "<value>",
  "propertyId": "6a8e87c256ffd64b71414aa6",
  "remarks": "Reviewed and approved",
  "targetType": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Create promotion request",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Promotion/promotionRoutes.js -> controllers/Promotion/promotionV2Controller.js

PATCH /api/v1/promotions/:id/approve

Bearer token

Purpose: Approve promotion request. (approvePromotionRequest)

Path params: id

Request body:

```
{
  "remarks": "Reviewed and approved"
}
```

Success response:

```
{
  "success": true,
  "message": "Approve promotion request",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Promotion/promotionRoutes.js -> controllers/Promotion/promotionV2Controller.js

PATCH /api/v1/promotions/:id/reject

Bearer token

Purpose: Reject promotion request. (rejectPromotionRequest)

Path params: id

Request body:

```
{
  "remarks": "Reviewed and approved"
}
```

Success response:

```
{
  "success": true,
  "message": "Reject promotion request",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Promotion/promotionRoutes.js -> controllers/Promotion/promotionV2Controller.js

POST /api/v1/promotions/maintenance/expire

Bearer token

Purpose: Expire promotions. (expirePromotions)

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Expire promotions",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Promotion/promotionRoutes.js -> controllers/Promotion/promotionV2Controller.js

GET /api/v1/promotions/my

Bearer token

Purpose: Get my promotion requests. (getMyPromotionRequests)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Promotion/promotionRoutes.js -> controllers/Promotion/promotionV2Controller.js

POST /api/v1/promotions/partner/request

Bearer token

Purpose: Create promotion request. (createPromotionRequest)

Query params: \$or, limit, page, partnerId, partnerMongold, promotionType, propertyId, propertyMongold, search, status, targetType

Request body:

```
{
  "targetId": "6a8d62970d52efcce682f351",
  "productCode": "PARTNER_BOOST"
}
```

Success response:

```
{
  "success": true,
  "message": "Create promotion request",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Promotion/promotionRoutes.js -> controllers/Promotion/promotionV2Controller.js

Purpose: Create promotion request. (createPromotionRequest)

Query params: \$or, limit, page, partnerId, partnerMongoid, promotionType, propertyId, propertyMongoid, search, status, targetType

Request body:

```
{
  "targetId": "6a8e87c256ffd64b71414aa6",
  "productCode": "PROPERTY_BOOST"
}
```

Success response:

```
{
  "success": true,
  "message": "Create promotion request",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Promotion/promotionRoutes.js -> controllers/Promotion/promotionV2Controller.js

Properties

GET /api/v1/properties

Public

Purpose: Get filtered live properties. (getFilteredLiveProperties)

Query params: \$or, bathrooms, bedrooms, category, city, furnishing, limit, maxPrice, minPrice, negotiable, page, price, propertyType, search, transactionType

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

POST /api/v1/properties

Bearer token

Purpose: Create property. (createProperty)

Request body:

```
{
  "amenities": "<value>",
  "category": "Residential",
  "city": "Indore",
  "creatorId": "<value>",
  "creatorRole": "<value>",
  "negotiable": "<value>",
  "price": "<value>",
  "status": "In_Progress",
  "tags": "<value>",
  "title": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Create property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

GET /api/v1/properties/:id

Public

Purpose: Get admin property by id. (getAdminPropertyById)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

PATCH /api/v1/properties/:id

Bearer token

Purpose: Update property. (updateProperty)**Path params:** id**Request body:**

```
{
  "existingImages": "<value>",
  "status": "In_Progress",
  "statusRemarks": "<value>",
  "updatedByName": "<value>",
  "updatedByRole": "<value>",
  "updatedByUserId": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Update property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js**PATCH** /api/v1/properties/:id/boost

Bearer token

Purpose: Boost property. (boostProperty)**Path params:** id**Request body:**

```
{
  "boostType": "<value>",
  "days": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Boost property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js**DELETE** /api/v1/properties/:id/delete

Bearer token

Purpose: Delete property. (deleteProperty)**Path params:** id**Request body:** None.**Success response:**

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

PATCH /api/v1/properties/:id/status

Bearer token

Purpose: Update property status. (updatePropertyStatus)

Path params: id

Request body:

```
{
  "notes": "<value>",
  "rejectionReason": "<value>",
  "status": "In_Progress"
}
```

Success response:

```
{
  "success": true,
  "message": "Update property status",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

PATCH /api/v1/properties/:id/unboost

Bearer token

Purpose: Remove property boost. (removePropertyBoost)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Remove property boost",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

GET /api/v1/properties/admin

Bearer token

Purpose: Get admin properties. (getAdminProperties)

Query params: \$or, boosted, partnerId, search, status

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

GET /api/v1/properties/all

Bearer token

Purpose: Get all properties. (getAllProperties)

Query params: \$or, city, role, search, status

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

GET /api/v1/properties/boosted

Bearer token

Purpose: Get boosted properties. (getBoostedProperties)

Query params: limit

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/V1/publicAppRoutes.js -> controllers/V1/homeController.js

GET /api/v1/properties/categories

Bearer token

Purpose: Get property categories. (getPropertyCategories)

Query params: category, limit, tab

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/V1/publicAppRoutes.js -> controllers/V1/homeController.js

GET /api/v1/properties/explore-nearby

Bearer token

Purpose: Explore nearby. (exploreNearby)

Query params: propertyId, radius

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/V1/publicAppRoutes.js -> controllers/V1/nearbyController.js

GET /api/v1/properties/filter

Public

Purpose: Get filtered live properties. (getFilteredLiveProperties)

Query params: \$or, bathrooms, bedrooms, category, city, furnishing, limit, maxPrice, minPrice, negotiable, page, price, propertyType, search, transactionType

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

GET /api/v1/properties/new-listings

Bearer token

Purpose: Get new listings. (getNewListings)

Query params: limit

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/V1/publicAppRoutes.js -> controllers/V1/homeController.js

GET /api/v1/properties/partner/:partnerId

Bearer token

Purpose: Get properties by partner. (getPropertiesByPartner)

Path params: partnerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyRoutes.js -> controllers/Property/propertyController.js

Property Publishing

GET /api/v1/property-publishing/:id/final-review

Admin Bearer

Purpose: Get final review property by id. (getFinalReviewPropertyById)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyPublishingRoutes.js -> controllers/Property/propertyPublishingController.js

PATCH /api/v1/property-publishing/:id/make-live

Admin Bearer

Purpose: Make property live after final review. (makePropertyLiveAfterFinalReview)

Path params: id

Request body:

```
{
  "notes": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Make property live after final review",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Property/propertyPublishingRoutes.js -> controllers/Property/propertyPublishingController.js

GET /api/v1/property-publishing/live

Admin Bearer

Purpose: Get live publishing properties. (getLivePublishingProperties)

Query params: \$or, category, city, search, transactionType

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyPublishingRoutes.js -> controllers/Property/propertyPublishingController.js

GET /api/v1/property-publishing/ready

Admin Bearer

Purpose: Get ready for final review properties. (getReadyForFinalReviewProperties)

Query params: \$or, category, city, search, transactionType

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyPublishingRoutes.js -> controllers/Property/propertyPublishingController.js

Purpose: Get property publishing summary. (getPropertyPublishingSummary)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Property/propertyPublishingRoutes.js -> controllers/Property/propertyPublishingController.js

Saved Properties

POST /api/v1/saved-properties

Bearer token

Purpose: Save property. (saveProperty)

Request body:

```
{
  "buyerId": "6a86b08e6007083b597138c1",
  "propertyId": "6a8e87c256ffd64b71414aa6"
}
```

Success response:

```
{
  "success": true,
  "message": "Save property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/buyer/savedPropertyRoutes.js -> controllers/Buyer/savedPropertyController.js

DELETE /api/v1/saved-properties/:buyerId/:propertyId

Bearer token

Purpose: Remove saved property. (removeSavedProperty)

Path params: buyerId, propertyId

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Resource deleted successfully"
}
```

Source: routes/buyer/savedPropertyRoutes.js -> controllers/Buyer/savedPropertyController.js

GET /api/v1/saved-properties/buyer/:buyerId

Bearer token

Purpose: Get buyer saved properties. (getBuyerSavedProperties)

Path params: buyerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/buyer/savedPropertyRoutes.js -> controllers/Buyer/savedPropertyController.js

GET /api/v1/saved-properties/check/:buyerId/:propertyId

Bearer token

Purpose: Check saved property. (checkSavedProperty)

Path params: buyerId, propertyId

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Check saved property",
  "data": "created or updated resource / operation result"
}
```

Source: routes/buyer/savedPropertyRoutes.js -> controllers/Buyer/savedPropertyController.js

Sellers

GET /api/v1/sellers

Bearer token

Purpose: Get all sellers. (getAllSellers)

Query params: \$or, city, isVerified, search, verified

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerController.js

GET /api/v1/sellers/:id

Bearer token

Purpose: Get seller by id. (getSellerById)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerController.js

GET /api/v1/sellers/:id/properties

Bearer token

Purpose: Get seller properties. (getSellerProperties)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerController.js

GET /api/v1/sellers/:id/summary

Bearer token

Purpose: Get seller summary. (getSellerSummary)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerController.js

GET /api/v1/sellers/:sellerId/properties/:propertyId

Bearer token

Purpose: Get seller property by id. (getSellerPropertyById)

Path params: propertyId, sellerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerController.js

POST /api/v1/sellers/applications/register

Public

Purpose: Create seller application. (createSellerApplication)

Request body:

```
{
  "name": "Seller Demo",
  "email": "seller@example.com",
  "phone": "9876500001",
  "city": "Indore",
  "state": "Madhya Pradesh",
  "country": "India",
  "address": "Vijay Nagar",
  "pinCode": "452010",
  "latitude": 22.7533,
  "longitude": 75.8937
}
```

Success response:

```
{
  "success": true,
  "message": "Create seller application",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerApplicationController.js

POST /api/v1/sellers/applications/resend-email-otp

Public

Purpose: Resend seller email otp. (resendSellerEmailOtp)

Request body:

```
{
  "applicationId": "SELLER_MONGODB_ID"
}
```

Success response:

```
{
  "success": true,
  "message": "Resend seller email otp",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerApplicationController.js

POST /api/v1/sellers/applications/resend-phone-otp

Public

Purpose: Resend seller phone otp. (resendSellerPhoneOtp)**Request body:**

```
{
  "applicationId": "SELLER_MONGODB_ID"
}
```

Success response:

```
{
  "success": true,
  "message": "Phone OTP resent",
  "data": {
    "applicationId": "SELLER_MONGODB_ID",
    "phone": "9876500001",
    "otp": "123456",
    "expiresIn": "10 minutes"
  }
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerApplicationController.js**POST /api/v1/sellers/applications/verify-email**

Public

Purpose: Verify seller email otp. (verifySellerEmailOtp)**Request body:**

```
{
  "applicationId": "SELLER_MONGODB_ID",
  "otp": "123456"
}
```

Success response:

```
{
  "success": true,
  "message": "Verify seller email otp",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerApplicationController.js**POST /api/v1/sellers/applications/verify-phone**

Public

Purpose: Verify seller phone otp. (verifySellerPhoneOtp)**Request body:**

```
{
  "applicationId": "SELLER_MONGODB_ID",
  "otp": "123456"
}
```

Success response:

```
{
  "success": true,
  "message": "Verify seller phone otp",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerApplicationController.js

PATCH /api/v1/sellers/auth/change-password

Bearer token

Purpose: Change seller password. (changeSellerPassword)**Request body:**

```
{
  "currentPassword": "<value>",
  "newPassword": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Change seller password",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerAuthController.js**POST** /api/v1/sellers/auth/login

Public

Purpose: Seller login. (sellerLogin)**Request body:**

```
{
  "email": "seller@example.com",
  "password": "TEMP_PASSWORD_FROM_EMAIL"
}
```

Success response:

```
{
  "success": true,
  "message": "Login successful",
  "token": "JWT_TOKEN",
  "data": {
    "id": "MONGODB_OBJECT_ID",
    "role": "buyer|seller|partner|admin"
  }
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerAuthController.js**POST** /api/v1/sellers/auth/login-with-otp

Public

Purpose: Seller login with otp. (sellerLoginWithOtp)**Request body:**

```
{
  "email": "user@example.com",
  "otp": "123456"
}
```

Success response:

```
{
  "success": true,
  "message": "Login successful",
  "token": "JWT_TOKEN",
  "data": {
    "id": "MONGODB_OBJECT_ID",
    "role": "buyer|seller|partner|admin"
  }
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerAuthController.js

Purpose: Send seller login otp. (sendSellerLoginOtp)

Request body:

```
{
  "email": "user@example.com"
}
```

Success response:

```
{
  "success": true,
  "message": "Login successful",
  "token": "JWT_TOKEN",
  "data": {
    "id": "MONGODB_OBJECT_ID",
    "role": "buyer|seller|partner|admin"
  }
}
```

Source: routes/Seller/sellerRoutes.js -> controllers/Seller/sellerAuthController.js

Team Partners

GET /api/v1/team-partners

Bearer token

Purpose: Get team partners. (getTeamPartners)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/teamPartnerRoutes.js -> controllers/Agent/teamPartnerController.js

POST /api/v1/team-partners/:ownerId/members

Bearer token

Purpose: Add team member. (addTeamMember)

Path params: ownerId

Request body:

```
{
  "name": "Sub Partner",
  "email": "subpartner@example.com",
  "phone": "9876543299",
  "identityDocuments": [
    {
      "documentType": "AADHAAR",
      "frontUrl": "https://example.com/front.jpg",
      "backUrl": "https://example.com/back.jpg"
    }
  ]
}
```

Success response:

```
{
  "success": true,
  "message": "Add team member",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/teamPartnerRoutes.js -> controllers/Agent/teamPartnerController.js

PATCH /api/v1/team-partners/:ownerId/members/:memberId/allocate-credits

Bearer token

Purpose: Allocate team credits. (allocateTeamCredits)

Path params: memberId, ownerId

Request body:

```
{
  "approvalThreshold": "<value>",
  "credits": 100
}
```

Success response:

```
{
  "success": true,
  "message": "Allocate team credits",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/teamPartnerRoutes.js -> controllers/Agent/teamPartnerController.js

GET /api/v1/team-partners/:ownerId/members/:memberId/credit-history

Bearer token

Purpose: Get team member credit history. (getTeamMemberCreditHistory)

Path params: memberId, ownerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/teamPartnerRoutes.js -> controllers/Agent/teamPartnerController.js

GET /api/v1/team-partners/:ownerId/properties

Bearer token

Purpose: Get team assigned properties. (getTeamAssignedProperties)

Path params: ownerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/teamPartnerRoutes.js -> controllers/Agent/teamPartnerController.js

PATCH /api/v1/team-partners/:ownerId/properties/:propertyId/delegate

Bearer token

Purpose: Delegate property to sub agent. (delegatePropertyToSubAgent)

Path params: ownerId, propertyId

Request body:

```
{
  "subPartnerId": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Delegate property to sub agent",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/teamPartnerRoutes.js -> controllers/Agent/teamPartnerController.js

PATCH /api/v1/team-partners/:ownerId/properties/:propertyId/remove-delegation

Bearer token

Purpose: Remove sub agent property delegation. (removeSubAgentPropertyDelegation)

Path params: ownerId, propertyId

Request body: None.

Success response:

```
{
  "success": true,
  "message": "Remove sub agent property delegation",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Agent/teamPartnerRoutes.js -> controllers/Agent/teamPartnerController.js

Purpose: Get verified team members. (getVerifiedTeamMembers)

Path params: ownerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Agent/teamPartnerRoutes.js -> controllers/Agent/teamPartnerController.js

User

GET /api/v1/user/dashboard-header

Bearer token

Purpose: Get dashboard header. (getDashboardHeader)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/V1/publicAppRoutes.js -> controllers/V1/homeController.js

Visits

GET /api/v1/visits/:id

Bearer token

Purpose: Get visit by id. (getVisitById)

Path params: id

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Visit/visitRoutes.js -> controllers/Visit/visitController.js

PATCH /api/v1/visits/:id/review

Bearer token

Purpose: Review visit request. (reviewVisitRequest)

Path params: id

Request body:

```
{
  "action": "approve",
  "approvedVisitAt": "2026-09-20T11:30:00.000Z",
  "adminRemarks": "Visit approved"
}
```

Success response:

```
{
  "success": true,
  "message": "Review visit request",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Visit/visitRoutes.js -> controllers/Visit/visitController.js

PATCH /api/v1/visits/:id/status

Bearer token

Purpose: Update visit status. (updateVisitStatus)

Path params: id

Request body:

```
{
  "status": "Completed",
  "partnerRemarks": "Visit completed successfully"
}
```

Success response:

```
{
  "success": true,
  "message": "Update visit status",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Visit/visitRoutes.js -> controllers/Visit/visitController.js

PATCH /api/v1/visits/:id/team-review

Bearer token

Purpose: Review sub agent visit by team owner. (reviewSubAgentVisitByTeamOwner)

Path params: id

Request body:

```
{
  "action": "approve",
  "remarks": "Reviewed and approved",
  "teamOwnerId": "<value>"
}
```

Success response:

```
{
  "success": true,
  "message": "Review sub agent visit by team owner",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Visit/visitRoutes.js -> controllers/Visit/visitController.js

GET /api/v1/visits/admin

Bearer token

Purpose: Get admin visits. (getAdminVisits)

Query params: approvalStatus, buyerId, limit, location, page, partnerId, propertyId, search, status, tab

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Visit/visitRoutes.js -> controllers/Visit/visitController.js

GET /api/v1/visits/partner/:partnerId

Bearer token

Purpose: Get visits by partner. (getVisitsByPartner)

Path params: partnerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Visit/visitRoutes.js -> controllers/Visit/visitController.js

POST /api/v1/visits/request

Bearer token

Purpose: Create visit request. (createVisitRequest)

Request body:

```
{
  "propertyId": "6a8ead9b58a6bb8fab38ebc1",
  "buyerId": "6a86b08e6007083b597138c1",
  "partnerId": "6a8d62970d52efcce682f351",
  "requestedVisitAt": "2026-09-20T11:30:00.000Z",
  "requestNotes": "Buyer wants a site visit."
}
```

Success response:

```
{
  "success": true,
  "message": "Create visit request",
  "data": "created or updated resource / operation result"
}
```

Source: routes/Visit/visitRoutes.js -> controllers/Visit/visitController.js

GET /api/v1/visits/summary

Bearer token

Purpose: Get visit summary. (getVisitSummary)

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Visit/visitRoutes.js -> controllers/Visit/visitController.js

GET /api/v1/visits/team/:ownerId/pending

Bearer token

Purpose: Get team pending visit requests. (getTeamPendingVisitRequests)

Path params: ownerId

Request body: None.

Success response:

```
{
  "success": true,
  "count": "number when applicable",
  "data": "object or array returned by controller"
}
```

Source: routes/Visit/visitRoutes.js -> controllers/Visit/visitController.js

6. Frontend implementation checklist

1. Set `API_BASE_URL` to `/api/v1`. 2. Store JWT securely and attach Bearer token through one Axios interceptor. 3. Do not attach Content-Type manually for multipart uploads. 4. Handle 401 by clearing the session; handle 403 as insufficient role/permission. 5. Use `MongoDB_id` for `:id` unless the endpoint explicitly names `partnerId`, `sellerId`, `buyerId` or `propertyId`. 6. Treat OTP values in responses as test-only and remove them before production. 7. Read pagination/count fields defensively because older controllers are not fully uniform.

Known engineering notes

- Generic auth is now buyer/tenant-only, preventing seller/partner onboarding bypass.
- Duplicate `CreditTransaction` `idempotencyKey` schema index warning remains unrelated to API auth.
- Legacy `/api` aliases are retained until the stated sunset date.
- Public allow-list is centralized in `middleware/apiAccessControl.js`.
- Some controllers still accept actor/resource IDs from request bodies; future hardening should derive these IDs from `req.user` and enforce ownership.